

SQIsign v3: Specification Review

Deirdre Connolly

Selkie Cryptography

durumcrustulum@gmail.com

7 September 2026 (draft for discussion)

Notes on the 1 September 2026 v3 specification [1] and the round-3 reference implementation [2], with suggested changes for the SQIsign team.

The interval-sampling defect is the main confirmed security finding. The hint-distribution concern remains provisional. Neither establishes a signature forgery or key-recovery attack. Part I sets out those findings and the related deployment considerations. Parts II and III collect the implementation corrections, ordered by conformance priority: *blocking* for fundamental algorithm or random-stream mismatches, then high, medium, and low. These priorities describe implementation impact, separately from the security severity in Part I. Part IV records the 42 v2 findings.

Contents

I	Security findings and deployment	3
1	Biased interval sampling	3
2	Hint degree weighting	4
3	Deployment properties to specify	5
II	Implementation conformance	6
4	Incorrect norm equation in LoopTwo	6
5	Sampling gates and ideal canonicalization	6
6	Random-byte consumption is underspecified	7
7	Incorrect product-ideal basis	7
8	Missing response basis point and reversed outputs	8
9	Incorrect power-of-two ideal formulas	8
10	Inconsistent challenge-kernel coordinates	9
11	Hint exponents and the base-curve basis	9
12	Wrong symmetric element in coordinate conversion	9
13	Conjugation and splitting in coprime ideal sampling	10

14	Response draw skipping and iteration policy	10
15	Checked curve decoding and verifier failures	11
16	Make deferred kernel checks traceable	11
17	Ideal-to-isogeny kernel layout	11
18	Parity loss in ideal intersection	12
19	Prime-norm sampling order and square-root choice	12
20	Primality-test witnesses and iteration counts	12
21	Key generation and signing failure handling	13
22	Theta-to-Montgomery candidates and point maps	13
23	The v3 square-root convention	13
III	Further specification clarifications	14
24	Carry the difference point through basis actions	14
25	Montgomery normalization and point transport	14
26	Work bounds for zero-hint basis reconstruction	14
27	Undefined batch normalization in change of basis	15
28	Tie-breaking in Minkowski reduction	15
29	The gluing ladder's output annotation	15
30	Incorrect fourth vector in the dual-lattice input	16
31	Unit-normalization ties	16
32	A dual-isogeny mismatch in the Kani matrix	16
33	Secret-key decoding bounds	16
34	Editorial corrections	16
IV	Disposition of the v2.0.1 findings	17
	Checks and source material	19
	References	19

Part I

Security findings and deployment

1 Biased interval sampling

Algorithm 4.24 and Section 4.1.4

MEDIUM

SAMPLEFROMINTERVAL sets $m = b - a + 1$ and $n = \text{ibz_bitsize}(m)$, draws an n -bit integer x , and accepts it when

$$x < m \left\lceil \frac{2^n}{m} \right\rceil.$$

The threshold is at least 2^n , so every draw is accepted. Reducing x modulo m therefore biases the output whenever m is not a power of two. For $[a, b] = [0, 2]$, the equiprobable draws 0, 1, 2, 3 produce 0, 1, 2, 0: the output probabilities are $1/2, 1/4, 1/4$, rather than $1/3$ each.

More generally, write $R = 2^n = qm + r$, with $0 \leq r < m$. The first r residues have $q + 1$ preimages and the others have q . Their total variation distance from uniform is

$$\Delta = \frac{r(m - r)}{mR}.$$

The prose describes 64 extra random bits, which line 2 omits. Extra bits would reduce this bias, but would not fix the acceptance test.

For the EIBox values in Table 9, the coefficient draws used by Algorithm 4.78 have the following distributions before later rejections:

Level	Interval	Distance from uniform	Mean (uniform: zero)
I	$[-15, 15]$	$15/496 \approx 3.02\%$	$-15/32$
III	$[-19, 19]$	$175/1248 \approx 14.02\%$	$-175/64$
V	$[-22, 22]$	$247/1440 \approx 17.15\%$	$-247/64$

These are biases in internal draws, not measured biases in public keys or signatures. They do not describe the C reference's wider sampler either.

Why this matters to the analysis. The response path is $\text{SIGN} \rightarrow \text{SAMPLEQUATERNIONRESPONSE} \rightarrow \text{RANDOMBOUNDEDELEMENT} \rightarrow \text{SAMPLEFROMINTERVAL}$. The sampler also feeds prime-norm ideal sampling and auxiliary ideal sampling through Algorithms 4.71 and 4.82.

Uniform coordinate sampling in a box enclosing a lattice ball, followed by rejection to the ball and to odd norm, gives equal weight to the accepted lattice points. Biased coordinate sampling loses that argument. The weights can depend on the chosen basis, whose representation can depend on secret ideal data. I have not established whether that dependence is visible in complete signatures; correcting the sampler restores the intended starting point for the distribution argument.

The accompanying script also gives a small $\mathbb{Z}^4$ example where odd-norm rejection and identifying v with $-v$ leave unequal weights. Those operations are therefore not a general cure for the bias, although this example is not a SQIsign response lattice.

Suggested change. Use an acceptance range whose size is divisible by m : replace the ceiling with a floor. An exact sampler is:

```

m = b - a + 1
if m == 1: return a
n = bit_length(m) + kappa
R = 2^n
limit = R - (R mod m)
repeat:
    x = uniform_n_bit_integer(prng)
until x < limit
return a + (x mod m)

```

Here $\kappa \geq 0$ changes the rejection rate, not the uniformity of accepted outputs. If approximately uniform fixed-draw sampling is preferred, I suggest specifying its statistical-distance budget and accounting for all calls and subsequent conditioning in the analysis. The prose, pseudocode, and reference should agree on which contract is intended.

This pseudocode specifies the distribution only. Public working-width bounds, observable retries, and failure handling need a separate timing argument. A fixed candidate batch with masked selection is one possible approach, provided its all-rejected probability and failure behavior are accounted for.

Reference implementation and conformance. The C reference uses a tracked bit bound plus 64 bits [3]. It adds m to the draw, checks that the sum still fits, and then reduces modulo m . In a release build the accepted sample space has size $R - m$; with assertions enabled the exceptional branch asserts. Since $R - m$ is not generally a multiple of m , this is not exact uniform sampling either, though the bias is much smaller than in the printed algorithm.

Our current Rust interval sampler already uses exact rejection over $[0, b - a]$. I suggest preserving that distribution during the port, while addressing its variable-time behavior separately. A changed random-byte schedule may require new KATs without changing the wire format or verification relation. Useful conformance cases include singleton and signed intervals, powers of two and adjacent widths, rejection boundaries, and the three EIBox intervals.

2 Hint degree weighting

Figure 4; Problem 8.1.5 and Section 8.2.2.2

Low (provisional)

In Figure 4, H_{unif} samples an odd degree $d \leq D_{\text{rsp}}$ with weight

$$\psi(d) = \prod_{\ell^e \parallel d} (\ell + 1) \ell^{e-1},$$

then samples a degree- d isogeny with non-cyclic kernels allowed. For d coprime to p , $\psi(d)$ counts cyclic kernels. Counting all degree- d kernels instead gives

$$\sigma_1(d) = \sum_{a|d} a = \prod_{\ell^e \parallel d} (1 + \ell + \cdots + \ell^e).$$

Here isogenies are counted up to isomorphism of the target, without quotienting by automorphisms of the source. Section 5.6 of the earlier security proof also identifies ψ as the cyclic count [4].

For example, degree 9 has twelve cyclic kernels and one additional kernel, $E[3]$, for a total of thirteen. Degree 25 similarly gives 30 versus 31. The printed procedure assigns an individual degree- d isogeny probability proportional to $\psi(d)/\sigma_1(d)$, which varies with d . Thus it is not uniform

over all outgoing bounded odd-degree isogenies. This is a well-defined distribution, but its degree dependence needs to be reconciled with the intended comparison to H_{sim} .

The latter first chooses a target from the stationary distribution, then samples uniformly among bounded odd-degree isogenies to that target. `RANDOMBOUNDEDELEMENT` does not impose primitivity or remove odd scalar factors, so we cannot simply assume that every response is cyclic.

A statistic worth checking. Consider $3 \mid \deg(\varphi_2)$. If the response homomorphisms are uniform modulo 3 in $M_2(\mathbb{F}_3)$, this event has probability

$$\frac{\#\{M : \det M = 0\}}{3^4} = \frac{33}{81} = \frac{11}{27}.$$

The ψ -weighted degree distribution, over increasing odd-degree bounds, instead gives limiting probability $2/5$. The gap is $1/135$. The event can be checked from the degree, or from the rank of the action on 3-torsion when the hint representation makes that action available.

The missing step is a quantitative residue-distribution argument for H_{sim} at the actual joint growth of p and D_{rsp} , including target conditioning and exceptional curves. The script checks the kernel counts, the 81 small-field matrices, and finite degree-weight marginals. It does not simulate H_{sim} or prove this distinguisher.

Suggested follow-up. If the intended measure is uniform over all outgoing bounded odd-degree isogenies, I suggest investigating $\sigma_1(d)$ as the degree weight and rechecking the resulting proof distribution and its transport properties. If the ψ weights are intentional, an explanation of the degree dependence and the statistic above would help settle the question.

Theorem 8.1.7 already includes a hint-distinguishing advantage term. This observation therefore questions whether the justification makes that term small; it does not contradict the reduction as written. Nor does it refute the separate argument using the hardness of the H_{sim} -assisted relation directly. I would treat this initially as a change or clarification to the analysis. Forcing primitive or cyclic responses in the signer would be a different protocol change requiring its own analysis.

3 Deployment properties to specify

The documented timing and binding limitations below affect deployment choices. The verifier and work-bound requests connect to the detailed implementation findings later in this review.

Timing resistance. Section 8.5 already explains that signing and key generation are not fully constant-time. For a hardened version, it would help to state which widths, iteration bounds, and failure behavior are public, and which operations remain outside the timing claim. In particular, correcting interval sampling does not make `IDEALTOISOGONY` or the whole signer constant-time. This needs resolution before deployment wherever the adversary can observe the relevant secret-dependent behavior.

Stronger message binding. The optional construction in Section 8.4.1 would benefit from explicit signing, verification, and encoding rules: carry the full 2λ -bit digest, use the designated λ bits for the challenge walk, and *compare all 2λ bits* during verification. Checking only the walk bits would discard the extra binding. This adds 16, 24, or 32 bytes at levels I, III, or V. Any incompatible profile needs a distinct format identifier and test vectors, including rejection when only the extra digest bits are changed.

As Section 8.4.2 explains, the auxiliary-isogeny representation remains malleable. The extended digest does not provide strong unforgeability, and matrix normalization does not make complete signatures unique.

Verifier preconditions and failure behavior. Section 3.1.2 already requires $A^2 \neq 4$ for ECCurve, and the C verifier checks the public and auxiliary coefficients before reconstructing bases [5]. I suggest making that checked decoding boundary explicit in Algorithm 3.24, along with propagation of supporting-routine failures to rejection. Similarly, map each deferred torsion, independence, kernel, and splitting check to the operation that enforces it. The inspected gluing code does check torsion and independence [6]; I have not established the suspected missing odd-response check as a bypass.

Sections 15 and 16 detail the verifier clarifications; §26 covers a justified work bound and failure policy for zero-hint basis reconstruction. Negative vectors for malformed encodings, singular coefficients, invalid kernel data, and failure propagation would make these requirements concrete.

Part II

Implementation conformance

4 Incorrect norm equation in LoopTwo

Affects Algorithm 4.148 (LOOPTwo) and Algorithm 4.149 (IDEALToISOGENYNORMEQUATION), hence every IDEALToISOGENY call. Conformance: **BLOCKING**

Line 10 sets $d \leftarrow 2^e - 2\mu^2 r_0 - \mu N$. The comment on line 11 says the target is $2^e - 2\mu^2 r_0 - \delta N$, and δ (lines 2 to 4) is otherwise unused. The reference (quat_qlapoty_loop_two, quaternion/ref/lv1x/qlapoty.c) uses δN with $\delta = 3$ when r_0 is even and 1 otherwise. With μN the value z of line 16 is not the one the Cornacchia step needs, and the returned β_1, β_2 do not satisfy $\text{nrd}(\beta_1) + \text{nrd}(\beta_2) = N2^e$.

Two further differences on the failure path:

- The reference iterates while $\mu \leq \lfloor \sqrt{2^e/r_0} \rfloor$. Before rounding and endpoint conventions, this upper bound is twice that of line 6 ($\mu < \sqrt{2^{e-2}/r_0}$).
- On LOOPTwo failure Algorithm 4.149 line 13 reruns LOOPONE. The reference (quat_qlapoty_normeq) reports failure and its caller (dim2id2iso_ideal_to_isogeny_qlapoty, id2iso/ref/lv1x/dim2id2iso.c) aborts.

Recommendation. Write δN on line 10. State the loop bound the reference uses and whether a rerun of LOOPONE is permitted.

5 Sampling gates and ideal canonicalization

Affects Algorithm 4.80 (SMALLESTEQUIVALENTIDEAL) via Algorithm 4.149 line 1, i.e. every IDEALToISOGENY call. Conformance: **BLOCKING**

The reference (quat_ideal_shortest_equivalent, quaternion/ref/lv1x/ideal.c) computes α_1 as the first vector of the reduced basis and $N' = \text{nrd}(\alpha_1)/N$, then:

1. It draws β only if $\text{gcd}(N', N) \neq 1$. Algorithm 4.80 always enters the sampling loop, and its loop condition is $\text{gcd}(N', \text{nrd}(\beta)/N) = 1$, a different test.
2. The generator is $\overline{\alpha_1}$, or $\beta \cdot \overline{\alpha_1}/N$ after a draw. Line 9 uses α_1 without conjugation.
3. After building $(N', (x, y))$ it canonicalizes: if $2x > N'$ then $(x, y) \leftarrow (N' - x, N' - y - 1)$ and the returned generator is multiplied by $\mathbf{i}$ on the right. Algorithm 4.80 has no such step.
4. Line 1 calls INERTIDEALREDUCTION without the bound β_λ that Algorithm 4.72 requires.

Item 1 means a literal Algorithm 4.80 consumes default-domain bytes on every call, shifting every later draw from that stream (Algorithm 4.23 witnesses, Algorithm 4.146). Item 3 replaces J by $J\mathbf{i}$,

which has the same codomain but a different image basis, so M_{sk} and M_{chl} can differ when that normalization changes the representative.

Recommendation. Specify the gated draw, the conjugate, and the $2x \leq N'$ canonical form. Explain which output representatives this normalization fixes, including the pushed basis; reproducing those choices is necessary for byte-for-byte conformance.

6 Random-byte consumption is underspecified

Algorithm 4.24 and callers in ideal and response sampling

Conformance: **BLOCKING**

Section 4.1.4 describes a draw of $\text{ibz_bitsize}(b - a + 1) + 64$ bits, while Algorithm 4.24 draws only $\text{ibz_bitsize}(b - a + 1)$ bits. The distributional defect and the exact rejection correction are discussed in §1. A separate conformance issue is the choice of working width and the resulting random-byte schedule.

The reference's `ibz_rand_interval_with_domain` draws $\lceil (b \rightarrow \text{bitlen} + 64) / 8 \rceil$ bytes and masks to $b \rightarrow \text{bitlen} + 64$ bits [3]. Here `bitlen` is the tracked bound of Definition 4.1.1, rather than the exact bit size of the endpoint or interval width. The reference then adds the interval width, checks that the sum fits, reduces, and shifts by a ; its overflow check is not the exact rejection rule recommended above. The spec does not give the tracked bound at each call site.

The earlier implementation comparison identified these byte schedules:

- The sign draw in Algorithm 4.71 uses $[0, 1]$ with the bound of the constant one, consuming nine bytes per candidate.
- Algorithm 4.83 samples $[0, 2\beta_i]$ and shifts by $-\beta_i$; its working width comes from the bound on $2\beta_i$ (`quat_lattice_sample_from_ball`, `quaternion/ref/1v1x/lattice.c`).
- Algorithm 4.78 uses `ibz_rand_interval_minm_m`, which samples $[0, 2m]$ with a 33-bit bound and subtracts m . For $m = 15$, each coefficient draw consumes thirteen bytes.
- Algorithm 4.46 assigns $\lceil \log_2 p \rceil$ as the endpoint bound.

Even a one-byte sign draw instead of nine bytes changes subsequent draws from that stream. A mathematically equivalent sampling interface therefore need not reproduce the reference KATs.

Suggested change. Specify the intended distribution first, then the public working-width bounds, PRNG domain, byte order, masking, singleton handling, and retry behavior needed for reproducibility. Tabulate the bounds at each call site, including the signed-interval variant. Do not substitute the exact size of a secret-dependent endpoint for its public bound without a timing argument. Updating the sampler and KATs can preserve the existing wire format and verification relation.

7 Incorrect product-ideal basis

Affects Algorithm 4.73 (MULIDEALBASIS) and Algorithm 4.75 (MULIDEALDUALREDUCTION) line 2, hence Algorithm 4.83 and every response.

Conformance: **HIGH**

Algorithm 4.74 builds $Y_1 = N_2(-u_2(2y_1 + 1) \bmod 2N_1)$ and $Y_2 = N_1(u_1(2y_2 + 1) \bmod 2N_2)$. Reducing modulo $2N_i$ preserves parity, so $Y_1 \equiv N_2u_2$ and $Y_2 \equiv N_1u_1 \pmod{2}$. With $u_1N_1 + u_2N_2 = 1$ and N_1, N_2 odd this gives $Y_1 - Y_2 \equiv Y_1 + Y_2 \equiv 1 \pmod{2}$.

Algorithm 4.73 prints the third column as $X_1 + X_2 + \frac{Y_1 - Y_2}{2} \mathbf{i} + \frac{\mathbf{i} + \mathbf{j}}{2}$. Its $\mathbf{j}$ -coefficient is $\frac{1}{2}$, so membership in $\mathcal{O}_0$ requires a half-integer $\mathbf{i}$ -coefficient. The printed coefficient $\frac{Y_1 - Y_2 + 1}{2}$ is an integer. The fourth column fails the same way: its $\mathbf{k}$ -coefficient is $\frac{1}{2}$ but its constant term $\frac{1 - Y_1 - Y_2}{2}$ is an integer. Neither column lies in $\mathcal{O}_0$, so the printed lattice is not the product ideal.

The reference (`quat_ideal_mul_00`) uses

$$X_1 + X_2 + \frac{Y_2 - Y_1}{2} \mathbf{i} + \frac{\mathbf{j}}{2}, \quad -\frac{Y_1 + Y_2}{2} + (X_2 - X_1) \mathbf{i} + \frac{\mathbf{k}}{2},$$

whose coefficients have the right parities. Equivalently, Algorithm 4.75 line 2 should read $A, B, C, E \leftarrow 2(X_1 + X_2), Y_2 - Y_1, -(Y_1 + Y_2), 2(X_2 - X_1)$. We confirmed on small primes that the reference's lattice equals $\overline{I}_1 \cdot I_2$, as Algorithm 4.74 promises, and that the printed one does not.

Recommendation. Correct the two columns in Algorithm 4.73 and line 2 of Algorithm 4.75.

8 Missing response basis point and reversed outputs

Affects Algorithm 3.21 (*HDRESPONSE*) and Algorithm 3.23 lines 17 and 18. Conformance: **HIGH**

1. e is used on lines 9 to 12 and never bound. The output types imply $e = e_{\text{rsp}}$.
2. Line 12 pushes $(P_{\text{com}}, 0)$ and $(Q_{\text{com}}, 0)$ only, but the outputs $\mathcal{B}_{\text{aux}}, \mathcal{B}_{\text{chl}}$ are typed as bases and Algorithm 3.23 line 18 feeds them to `SETCHANGEOfBASISMATRIX`, whose `CHANGEOfBASIS` calls need $R - S$. The reference (`compute_dim2_isogeny_challenge`, `signature/ref/lvlx/sign.c`) pushes P, Q , and $P - Q$. Recomputing the difference on the codomain is not an option, see Part IV, item 17.
3. By the convention of Algorithm 3.20 ($\deg \varphi_1 = q$ from E_1), the first codomain factor for $E_1 = E_{\text{com}}$ is E_{chl} and the second is E_{aux} . Line 12 destructures them as $E_{\text{aux}}, E_{\text{chl}}$. The reference reads E_{chl} from the first factor and E_{aux} from the second. A signer following line 12 encodes the challenge curve as E_{aux} and fails its own verifier.
4. Line 7 inverts $\text{IdealNorm}(I_{\text{sk,chl}})$ modulo 2^f ; the reference inverts modulo $2^{e_{\text{rsp}}+2}$. The results agree after the reduction on lines 10 and 11.
5. Line 12 writes $0_{E_{\text{com}}}$ for a point of E'_{aux} .

Recommendation. Bind e , push the third point, and swap the two curves on line 12.

9 Incorrect power-of-two ideal formulas

Affects Algorithm 4.68 (*INERTGENToIDEALPOWTwo*) via Algorithm 4.151 line 14, i.e. the challenge ideal of every signature. Conformance: **HIGH**

Line 10 reads $X \leftarrow t(ys + zy + yz) \bmod 2^e$. With s the inverse computed on line 9 this is not a formula for X . The reference (`quat_ideal_create_00_pow_two`, `quaternion/ref/lvlx/ideal.c`) computes, with $(t, x, y, z) = \text{COORDINATESO}_0(\gamma \bmod 2^e)$ and $s = (y^2 + z^2)^{-1} \bmod 2^e$,

$$X = s(ty + xz + yz) \bmod 2^e, \quad Y = s(xy - tz - z^2) \bmod 2^e.$$

The reference also normalizes parities before these formulas: after the $(\mathbf{i} + \mathbf{j})/2$ step of lines 3 to 5 it multiplies γ by $\mathbf{i}$ on the left whenever y is odd, so that the formulas see y even and z odd. Algorithm 4.68 accepts $(y, z) \bmod 2 \in \{(0, 1), (1, 0)\}$ and applies one formula to both.

Recommendation. Replace lines 10 and 11 with the formulas above and add the $\mathbf{i}$ normalization, or give formulas valid for both parity cases.

10 Inconsistent challenge-kernel coordinates

Affects Algorithm 3.16, Algorithm 4.151 (*KERNELDECOMPOSEDTOIDEAL*), and Algorithm 3.23 lines 12 and 13. Conformance: **HIGH**

Three statements about the input (c_1, c_2) disagree:

- Algorithm 3.16: both divisible by 2^{f-e} , not both by 2^{f-e+1} , and $E_0[I] = \langle [c_1]P_0 + [c_2]Q_0 \rangle$ on the 2^f -torsion basis $\mathcal{B}_0$.
- Algorithm 4.151: “a point $[c_1]P_0 + [c_2]Q_0$ of order 2^e ”.
- Algorithm 3.23 line 12: $(c_1, c_2) = M_{\text{sk}} \cdot [1, a_{\text{chl}}]^T$ with M_{sk} reduced modulo 2^λ , which are coordinates on the 2^λ -torsion basis $[2^{f-\lambda}] \mathcal{B}_0$ and are not divisible by $2^{f-\lambda}$.

The reference (`id2iso_kernel_dlogs_to_ideal_even`, `id2iso/ref/1v1x/id2iso.c`) takes the third reading: it evaluates M_{sk} on $(1, a_{\text{chl}})$ and reduces every intermediate modulo 2^λ . An implementor who follows Algorithm 3.16 multiplies by $2^{f-\lambda}$ first and computes the ideal of a different point. Algorithm 4.151 line 1 also uses M_j , which is not among the three precomputed matrices of §4.6; it equals $2M_{(i+j)/2} - M_i$.

Recommendation. State once that (c_1, c_2) are coordinates modulo 2^e on $[2^{f-e}] \mathcal{B}_0$, and fix Algorithm 3.16 to match.

11 Hint exponents and the base-curve basis

Affects Algorithm 3.11 (*TORSIONBASISTOHINT*), Algorithm 3.9 lines 1 and 2, Algorithm 4.94, Algorithm 4.95. Conformance: **HIGH**

Algorithm 3.11 says the hint decodes to the basis $([2^{e_h}]P, [2^{e_h}]Q)$ of $E[2^{e-e_h}]$, and Algorithm 4.94 types the output as Hint_{e-e_h} . Algorithm 3.9 calls *TORSIONBASISTOHINT*(E_1, f, e) with $e = e_{\text{rsp}} + 2$, which under Algorithm 3.11 would produce a hint for $E[2^{f-e}] = E[2^{126}]$. Algorithm 3.24 line 9 decodes the same hint at exponent $e_{\text{rsp}} + 2$.

Algorithm 4.94 lines 21 to 26 resolve it: e_h is the exponent at which *TORSIONBASISFROMHINT* will be called, and the adjustment selects between $[2^{e-e_h}](R \pm S)$ so that the third coordinate matches the one Algorithm 4.95 recomputes. The reference (`ec_curve_to_basis_2f_to_hint`, `ec/ref/1v1x/basis.c`) agrees. Only the prose of Algorithm 3.11 and the type in Algorithm 4.94 are wrong.

A second gap: when $A = 0$ the reference returns the precomputed basis of E_0 with hint 0, and *FROMHINT* likewise handles this curve separately. Algorithm 4.94 has no such branch. Its candidate has $x_R = 0$ for every h ; $(0, 0)$ is a point of order two on E_0 , and the construction gives $S = R$ before cofactor multiplication. It cannot produce the required basis. Hint 0 therefore covers both the base curve and a fallback search; the reference distinguishes them by testing A .

Recommendation. Rewrite Algorithm 3.11 as “return a basis of $E[2^e]$ and a hint that *TORSIONBASISFROMHINT*($E, e_h, \cdot$) decodes to a basis of $E[2^{e_h}]$ with the same third coordinate”. Add the $A = 0$ branch to Algorithm 4.94 and Algorithm 4.95.

12 Wrong symmetric element in coordinate conversion

Affects Algorithm 4.119 (*THETACHANGEOfCOORDINATES*) lines 14 to 21, hence every gluing.

Conformance: **HIGH**

Line 3 sets $(\mathbf{G}^1, \mathbf{G}^2, \mathbf{G}^3)$ to the symmetric elements of $K'_{1,1}$, $K'_{2,1}$, and their sum on E_1 . Lines 14 to 21 build rows 2 and 3 of $\mathbf{M}$ from $\mathbf{G}^2$, the element of $K'_{2,1}$. The reference (`gluing_change_of_coordinates`, `hd/ref/1v1x/gluing.c`) builds those rows from $\text{Basis}_1.g1$, the element of $K'_{1,1}$, that is $\mathbf{G}^1$. Row 1 uses $\mathbf{H}^2$ in both. The index pattern of the products is otherwise identical.

Recommendation. Replace G^2 by G^1 in lines 14 to 21, or explain why the two agree.

13 Conjugation and splitting in coprime ideal sampling

Affects Algorithm 4.79 (EQUIVALENTCOPRIMEIDEAL) via Algorithm 4.81 (both keys), Algorithm 3.18 line 1 (every signature). Conformance: **High**

The reference (quat_ideal_small_equivalent_coprime_enumeration, quaternion/ref/lv1x/ideal.c) differs from Algorithm 4.79:

1. Coefficients come from the $[-m, m]$ sampler variant (§6), 13 bytes each, and an all-zero vector is skipped before the content is divided out.
2. N' must be odd before the primality or coprimality test. For $M = 2$ oddness is the whole test.
3. β is conjugated before the ideal is built.
4. A non-inert $\chi_I(\beta)$ is not rejected. The reference calls quat_ideal_create_OO_odd, which splits off the $\mathbb{Z}[i]$ factor as in Algorithm 4.76 and returns the inert part. Algorithm 4.79 line 10 resamples instead.
5. The loop has no ElBoundIteration cap.

Item 3 changes which ideal is produced from a given β and item 4 changes how many draws are made before returning. Both alter I_{sk} and I_{com} on inputs that occur in the published KATs. Line 9 also passes the arguments of INERTGENTOIDEAL in the opposite order from Algorithm 4.70.

Recommendation. Reconcile the conjugation, oddness test, and non-inert splitting with the intended ideal distribution, then state one algorithm for implementations to follow. Specify whether the iteration bound is normative and how its failure policy relates to the analysis.

14 Response draw skipping and iteration policy

Affects Algorithm 4.83 (RANDOMBOUNDEDELEMENT) via Algorithm 3.18 line 5. Conformance: **High**

The reference (quat_lattice_sample_from_ball) differs from Algorithm 4.83 in its random-byte schedule and failure behavior:

1. When $\beta_i = 0$ it sets $x_i = 0$ without consuming randomness. Algorithm 4.83 line 7 always draws.
2. Each coordinate is drawn from $[0, 2\beta_i]$ and shifted, see §6.
3. The loop is unbounded; there is no BoundIterationRBE check.

The acceptance test is equivalent: the reference accepts when $x^t G x \not\equiv 0 \pmod{4}$ and $x^t G x \leq 2B$ with G the Gram matrix of twice the norm, which is M odd and $M \leq B$. The box formula agrees with line 6 (quat_lattice_bound_parallelogram, quaternion/ref/lv1x/111_applications.c). Line 5 reuses the loop index i of line 4.

Recommendation. State the skip on $\beta_i = 0$ and the interval passed to the sampler, using the corrected distribution in §1. Reconcile the loop policy with the failure analysis in §9.5: specify the cap, restart or error behavior, and its effect on the accepted distribution and timing. Removing the cap or introducing retries requires checking the assumptions of that analysis.

15 Checked curve decoding and verifier failures

Algorithm 3.24; §3.1.2, §3.4.1, §3.4.3

Conformance: [MEDIUM](#)

The reference rejects $A_{pk} = \pm 2$ and $A_{aux} = \pm 2$ before reconstructing bases [5]. The specification already requires $A^2 \neq 4$ for ECCurve in §3.1.2. The useful clarification is where decoding untrusted bytes establishes this invariant before routines that assume a valid curve are called; the missing line in VERIFY is not by itself evidence that the specification permits singular inputs.

Supporting calls on lines 5–11 may fail on malformed inputs. Algorithm 3.24 should explicitly route these failures through the exception convention of §3.4.1 to `false`. A caller should not need to infer whether a failed basis reconstruction, invalid kernel, or unsuccessful split is an API error or an invalid signature.

The supersingularity claim in §2.3.3 would also benefit from a short argument linking the order and kernel checks to the properties needed of the public and auxiliary curves. Section 3.4.3 expressly permits deferred validation; §16 asks for that dependency to be made explicit.

Suggested change. Show checked decoding of both coefficients before basis reconstruction, state that supporting-routine failures reject the signature, and identify the checks that justify each required curve and torsion property. Include negative vectors for singular coefficients and failed subroutines. No singular-curve acceptance or odd-response verification bypass has been established here.

16 Make deferred kernel checks traceable

Algorithm 4.116, Algorithm 4.118, Algorithm 4.122, Algorithm 4.124; §3.4.3

Conformance: [MEDIUM](#)

The reference gates some kernel validation on its `verify` flag. In particular, the gluing code explicitly verifies the projected two-torsion points and their independence through `verify_two_torsion` [6]. The printed GLUINGCODOMAIN does not expose the same standalone check. Related constraints also occur in supporting routines such as COSYMELEM, so the absence of that line alone does not establish a difference in the accepted signatures.

Section 3.4.3 allows checks to be deferred to operations that enforce the required property. What is missing for an implementor is a direct mapping from each required order, independence, and splitting condition to the operation that checks it, together with the treatment of failure. Similarly, Algorithm 4.116 states its zero-coordinate error without the reference’s verification-mode qualification.

Suggested change. List the verifier’s required predicates and cite the routine enforcing each one. Distinguish mandatory validation of attacker-provided data from signing-time consistency checks whose preconditions are already established. Add negative vectors that reach each rejection point. No signature accepted by the complete specified verifier but rejected by the reference has been demonstrated for this concern.

17 Ideal-to-isogeny kernel layout

Affects Algorithm 4.150 (IDEALTOISOGENY) lines 5 to 7 and 13, and Algorithm 4.149 line 19.

Conformance: [MEDIUM](#)

The pseudocode places (P_0, Q_0) on the first factor of $E_0 \times E_0$, the images $\theta(P_0), c_2\theta(Q_0)$ scaled by v_1 and $c_2 = v_1(1 - v_2 2^{f-2})$ on the second factor, with $\theta = \beta_2\beta_1/N$, and reads E_I from the first codomain factor. This matches Eq. (28) in Appendix A.

The reference does something else. `quat_qlapoty` returns $\bar{\theta} = \beta_1\bar{\beta}_2/N$ (it conjugates after the norm equation). `dim2id2iso_ideal_to_isogeny_qlapoty` then forms $T_1 = ([v_1]\bar{\theta}(P_0), P_0)$ and

$T_2 = ([v_1]\bar{\theta}(Q_0), Q_0)$, with no c_2 factor, runs a dedicated chain variant (`theta_chain_compute_and_eval_E2`), and reads E_I and the pushed basis from the *second* codomain factor.

Both recipes define the same $(2^{f-2}, 2^{f-2})$ -isogeny kernel $\langle [4]T_1, [4]T_2 \rangle$, and the outputs are x -only, so they may well produce identical bytes. We have not verified that a literal Algorithm 4.150 does, and the pair (T_1, T_2) the reference uses is not isotropic at level 2^f in the sense required by §4.5.4.1 (the Weil pairing exponent is $v_1 2^{f-2}$), so it relies on chain internals the spec does not describe.

Recommendation. State that Algorithm 4.150 was validated against the reference KATs, or document the reference’s kernel layout and the chain variant it needs.

18 Parity loss in ideal intersection

Affects Algorithm 4.77 (INTERSECT) via Algorithm 3.23 line 14 and Algorithm 3.21 line 4.

Conformance: [MEDIUM](#)

Line 2 reduces $X_2 + iY_2$ with $\text{MOD}_{O_0}(\cdot, N_2)$, which maps each coordinate to $[0, N_2)$. Lines 4 and 5 then divide X_2 and $Y_2 - u^2 - v^2$ by two and assert the results are integers. Reduction modulo the odd N_2 does not preserve parity, so the claimed integrality can fail after the reduction. The reference (`quat_ideal_intersect_O0`) reduces both coordinates modulo $2N_2$, which keeps the parities of the unreduced product.

Recommendation. Write line 2 with modulus $2N_2$, or reduce after the halving.

19 Prime-norm sampling order and square-root choice

Affects Algorithm 4.71 (PRIMENORMIDEALINERTSAMPLING) via Algorithm 4.81, i.e. I_{sk} and I_{com} .

Conformance: [MEDIUM](#)

Three details fix the bytes and are not in the text.

1. Draw order. The reference (`quat_random_ideal_O0_given_prime_norm`) draws x , loops until the residuosity test passes, computes the square root, and only then draws the sign bit. The pseudocode agrees, but nothing says the sign bit is a separate call to the interval sampler on $[0, 1]$, which costs nine bytes (§6).
2. Root choice. Line 5 calls `MODULARSQRT`, whose output for $N \equiv 1 \pmod{4}$ depends on the non-residue used by Tonelli–Shanks (Algorithm 4.20 does not fix it). The sign bit of line 9 does not absorb this: the two roots differ in Y , hence in y , before and after the sign flip.
3. The test in line 3 is implemented as $\left(\frac{-4x^2 - p \bmod N}{N}\right) = 1$ together with $(-4x^2 - p) \bmod 4 \in \{0, 1\}$.

This is equivalent, but the spec should say so since “square mod $4N$ ” has no algorithm attached.

Recommendation. Pin the root returned by `MODULARSQRT` (e.g. the smaller representative, or the output of a stated Tonelli–Shanks non-residue search), and state the byte cost of the sign draw.

20 Primality-test witnesses and iteration counts

Algorithm 4.23 and callers Algorithm 4.46, Algorithm 4.79, Algorithm 4.148 Conformance: [MEDIUM](#)

Line 18 samples a witness at random without naming a PRNG. The values of `reps` (64, 96, 128) appear in the prose of §4.1.3 but not in Tables 8–10. At level I the reference passes `QUAT_primality_num_iter = 64` and draws witnesses from the default domain (`precomp/ref/p324_3/include/quaternion_constants.h`).

Drawing witnesses from the key, com, or res stream instead changes the later draws in that stream. Isolating the witnesses in prng_{def} avoids this source of interference. The primality procedure is a probable-prime test: different witness choices can affect acceptance of composites. It should not be described as a Las Vegas computation whose result is independent of randomness.

Suggested change. Tabulate reps, name prng_{def} in the algorithm, and state the false-acceptance bound and assumptions used for this probable-prime test. Keep its error accounting distinct from the random-stream convention needed for conformance.

21 Key generation and signing failure handling

Affects Algorithm 3.22 (KEYGEN) and Algorithm 3.23 (SIGN).

Conformance: Low

The reference key generation (`protocols_keygen`, `signature/ref/lv1x/keygen.c`) loops the whole of lines 3 and 4 until `IDEALToISOGENY` succeeds, drawing fresh "key"-domain bytes each time. Signing (`protocols_sign`) returns failure on the two coprimality exceptions of Algorithm 3.18 and Algorithm 3.21 and on `RANDOMIDEALGIVENORM` failure, and aborts on `IDEALToISOGENYNORMEQUATION` failure. The spec raises and stops. Chapter 9 analyzes rare failures under its sampling assumptions, but a small failure probability does not specify the API contract. The distribution correction in §1 and the loop policy in §14 should be reconciled with those assumptions.

Recommendation. State the intended behavior on each exception.

22 Theta-to-Montgomery candidates and point maps

Affects Algorithm 4.126 (THETAToMONTGOMERY) and Algorithm 4.96.

Conformance: Low

The third theta null point enumerated is $(a + ib : a - ib)$ in the spec and $(ia + b : a + ib)$ in the reference (`ec_theta_to_montgomery`, `ec/ref/lv1x/normalize.c`). The two give opposite Montgomery coefficients, so after the $\pm A_i$ step of line 13 the same six values are compared and the same A_{max} is chosen. The conversion matrices N of lines 20 to 31 are indexed by the position of the maximum, and the positions of the third pair are swapped between the two lists. We did not validate the printed N table against the reference's.

Recommendation. Either enumerate the same six null points as the reference or state that any enumeration works and give the N matrices as functions of the chosen null point rather than of an index.

23 The v3 square-root convention

Affects Algorithm 4.35 (FP2SQRT) and every consumer of PROJECTIVEDIFFERENCE.

Conformance: Low

Algorithm 4.35 matches the reference's `fp2_sqrt` (`gf/ref/lv1x/fp2.c`) line for line, including the $x_1 = 0$ select. The v2 reference additionally negated the result to make the real part even; that step is gone and Algorithm 4.35 is the whole convention. Nothing in §1.4 says so. An implementor porting a v2 field layer keeps the parity step, and every torsion basis differs.

Recommendation. Note in §4.2.1.3 that the output of Algorithm 4.35 is not further canonicalized, and list the change in §1.4.

Part III

Further specification clarifications

24 Carry the difference point through basis actions

Affects Algorithm 3.7 (*ACTONBASIS*) and its callers Algorithm 4.150 lines 7 and 9, Algorithm 3.21 line 8, Algorithm 3.24 line 11. Conformance: [MEDIUM](#)

Algorithm 3.7 returns the pair $([x_1]P + [x_3]Q, [x_2]P + [x_4]Q)$. Algorithm 4.150 line 9 deconstructs a triple (P_1, Q_1, R_1) from it and pushes R_1 through the chain as the difference of the first two. In x -only arithmetic the third point is a separate biscalar multiplication by $(x_1 - x_2, x_3 - x_4)$ on the same triple, and it cannot be recovered from the first two outputs (v2 review, item 17). The reference's `id2iso_matrix_application_even_basis` computes it when asked.

Recommendation. Define *ACTONBASIS* on triples and state the third scalar pair. Restate the rule the v2 review asked for, which v3 still leaves implicit: once a basis $(P, Q, P - Q)$ exists, the third point is carried through every doubling, matrix action, and isogeny evaluation alongside P and Q . Recomputing it with *PROJECTIVEDIFFERENCE* picks a square-root branch independently and can return $P + Q$ in place of $P - Q$, invalidating later differential additions that rely on the original sign relation.

25 Montgomery normalization and point transport

Affects Algorithm 4.96 (*NORMALIZEMONTGOMERY*) and Algorithm 4.99 (*TWOISOGENYCHAIN*) line 26. Conformance: [MEDIUM](#)

1. Line 1 feeds $(X_P + Z_P, X_P - Z_P)$ of a 4-torsion point to *COMPUTEMONTCOEFF*, which expects a one-dimensional theta null point. The relation is Eq. (10) of §4.5.7.1 with $P = (a + b : a - b)$, thirty pages later, and is not cited.
2. *COMPUTEMONTCOEFF* raises when $C = 0$. Algorithm 4.96 does not say what happens then. The reference (`ec_theta_to_montgomery`) returns failure, which Algorithm 3.24 must treat as rejection.
3. Changing the Montgomery model moves every point. Algorithm 4.126 returns the 2×2 map $\mathbf{N}$; Algorithm 4.96 returns only the curve. Algorithm 4.99 evaluates pts on line 24 and normalizes on line 26, so its output points are on the pre-normalization model. No protocol call evaluates points through *TWOISOGENYCHAIN*, so this is latent, but the algorithm as printed is inconsistent. The reference's `ec_normalize_montgomery` applies the map to a basis when one is passed.

Recommendation. Cite Eq. (10) on line 1, add a failure clause, and return the point map (or state that callers must not evaluate points).

26 Work bounds for zero-hint basis reconstruction

Algorithm 4.95 lines 1–3 and the searches in Algorithm 4.94, reached from Algorithm 3.24 Conformance: [LOW](#)

A zero hint makes *TORSIONBASISFROMHINT* reconstruct the basis by searching again. Algorithm 4.94 does not give a worst-case number of candidates or a policy for search exhaustion. The reference also uses search loops in `ec/ref/1v1x/basis.c`. This leaves the maximum verification work and the treatment of an exhausted implementation counter unclear.

This is a request for a work bound and a consistent failure policy. We have not established an input that makes the v3 fallback search nonterminating. Fast searches on sampled curves do not establish a worst-case bound over the verifier’s permitted input domain.

Suggested change. Give a justified search bound or a bounded reconstruction procedure, and propagate its failure to signature rejection. Any cap that changes the verifier’s acceptance behavior needs an honest-input failure argument and a common conformance rule. Specify counter exhaustion explicitly, and test the last permitted candidate and failure path. Do not infer a safe cap from an empirical maximum alone. The separate $A = 0$ basis convention is addressed in §11.

27 Undefined batch normalization in change of basis

Affects Algorithm 4.106 (CHANGEOfBASIS) line 2 and Algorithm 4.104 (TATE). Conformance: **Low**

Line 2 calls NORMALIZE on ten points and the curve. No algorithm by that name exists. Algorithm 4.104 requires its three points as $(X : 1)$ and its curve as $(A_{24} : C_{24})$, and Algorithm 4.101 divides by C_{24} at every step. The intent is a batch inversion of the ten Z -coordinates (and presumably of C_{24}) with Algorithm 4.36.

Recommendation. Define NORMALIZE or inline the batch inversion.

28 Tie-breaking in Minkowski reduction

Affects Algorithm 4.65 (MINKOWSKIREDUCE) and the canonical-form claim of Algorithm 4.60.

Conformance: **Low**

Line 10 sorts the candidate list by norm. Candidates of equal norm have no stated order, and lines 13 to 25 take the first independent ones. §9.1 argues that equal minima occur with probability far below $2^{-\lambda}$, which covers the successive minima but not the 27-element candidate list, where several candidates share a norm routinely (for instance c and $-c$ are excluded by construction, but $(1, 1, 0, 0)$ and $(1, -1, 0, 0)$ are not). The output is claimed canonical up to a global sign.

Recommendation. Specify a total order on candidates (norm, then the coefficient vector lexicographically) or argue that the choice among equal-norm candidates does not change T .

29 The gluing ladder’s output annotation

Affects Algorithm 4.90 (GLUINGLADDER).

Conformance: **Low**

The output is annotated $[2^e - 1]P, [2^e]P$. Starting from $(R_0, R_1) = (Q, P)$ and applying xDBLADD e times returns $(R_1, R_0) = ([2^e]P, [2^e - 1]P + Q)$. That is what Algorithm 4.131 line 14 relies on, with $Q = 0$ on the first call and $Q = [2^m - 1]T$ afterwards.

Recommendation. Annotate the output as $([2^e]P, [2^e - 1]P + Q)$.

30 Incorrect fourth vector in the dual-lattice input

Affects Algorithm 4.60 (DUALLATREDUCTION4DZ).

Conformance: Low

The input is written as $2N\mathbb{Z} + 2Ni\mathbb{Z} + (A + Bi + j)\mathbb{Z} + (C + Ei + j)\mathbb{Z}$. Doubling the basis of Algorithm 4.73 gives a fourth vector with $\mathbf{k}$, not $\mathbf{j}$, and Algorithm 4.75 line 2 assigns C, E to the $\mathbf{k}$ -column. The Gram profile on line 2 of Algorithm 4.60 is right for $\mathbf{k}$.

31 Unit-normalization ties

Affects Algorithm 4.59 (LEHMERGLZI) lines 20 and 21, hence Algorithm 4.72 and §5.

Conformance: Low

Lines 20 and 21 pick $u \in \{\pm 1, \pm i\}$ so that the real and imaginary parts of uA (respectively uH) are nonnegative. When one part is zero two units qualify. The choice fixes the first reduced vector, which is α_1 in Algorithm 4.80, so it reaches the key bytes. Zero parts are rare but the rule should be total.

32 A dual-isogeny mismatch in the Kani matrix

Affects §4.6.1, Eq. (15), and Algorithm 3.20.

Conformance: Low

The diagram above Eq. (15) draws $\psi_2 : E' \rightarrow E_0$, but the $(2, 1)$ entry of $\Phi : E_0^2 \rightarrow E_I \times E'$ is written $-\psi_2$, which would need domain E_0 . §4.5.4 writes the same entry as $-\psi_2$ with $\psi_2 : E_4 \rightarrow E_1$ and the same problem. One of the arrow or the entry needs a hat.

33 Secret-key decoding bounds

Affects Chapter 6 (secret key encoding).

Conformance: Low

The inert ideal is encoded as three fields of `FP_ENCODED_BYTES` bytes. Nothing bounds $N = \text{IdealNorm}(I_{\text{sk}})$ by 2^{328} at NIST-I; the bound follows from Algorithm 4.79 and §9.3 but is not stated at the encoding. No parse-time checks are listed for a secret key (N odd, $0 \leq x, y < N$, M_{sk} entries below 2^λ and normalized). A signer that accepts a tampered key runs Algorithm 3.23 on it.

34 Editorial corrections

Low-severity text defects.

Conformance: Low

- Algorithms 4.42 to 4.48 are numbered out of order (4.43, 4.44, 4.45, 4.42, 4.47, 4.46, 4.48).
- Algorithm 4.49 line 6 calls `SUMOF SQUARES( $r, m, \cdot$ )`; Algorithm 4.52 takes (m, r, β_λ) .
- Algorithm 4.79 line 9 calls `INERTGENTOIDEAL( $\beta, N'$ )`; Algorithm 4.70 takes (N, γ) .
- Algorithm 4.80 line 1 omits the β_λ argument of Algorithm 4.72.
- Algorithm 4.83 line 5 reuses the loop index i of line 4.
- Algorithm 3.21 line 12 writes $0_{E_{\text{com}}}$ for a point of E'_{aux} .
- Algorithm 3.13 is declared on `INERTIDEAL` with a footnote that it “extends naturally”; Algorithm 4.83 line 3 applies it to a `MULIDEAL`. Say N for all three types.
- Table 5 gives $e_{\text{rsp}} = \lceil \log(p)/2 \rceil + \lambda/4 + 1$ and Table 8 gives $1 + \lceil \frac{1}{4}(\lambda + 2 \log_2 p) \rceil$. They agree because λ is a multiple of 8; one formula suffices.
- Algorithm 3.14 and Algorithm 3.15 write `cls(( $I$ ))`.
- §9.3.1: “quadratif”, “symetries du to”, “we’re gonna”.
- Algorithm 4.151 line 1 uses M_j , which is not precomputed in §4.6; give it as $2M_{(i+j)/2} - M_i$ or add it to §4.7.

Part IV

Disposition of the v2.0.1 findings

Each row cites the section of the v2.0.1 review [7]. *Resolved* means v3 addresses the original issue, and *moot* means the relevant construction was replaced or removed. *Partial* marks an addressed issue with a related concern elsewhere in v3. The zero-hint row is a request to clarify work bounds; it is not a confirmed nontermination defect. These statuses concern the original v2 finding, not every property of the associated v3 algorithm.

#	v2.0.1 finding	Status	v3.0 locus
1	$\mathbb{F}_{p^2}$ square root canonicalization	resolved	Algorithm 4.35 is a complete deterministic formula; the parity step is gone (§23).
2	PRODUCTToTHETA coordinate convention	resolved	Algorithm 4.121 lines 4 to 8.
3	“Squared theta” definition	resolved	§4.5.2.1 separates SQUARING and HADAMARD; every algorithm spells both out.
4	GLUINGEVAL cross-curve products	resolved	Algorithm 4.123 carries explicit subscripts (new formulas).
5	Projective inverse of the dual null point	resolved	Algorithm 4.110 and Algorithm 4.122 lines 15 to 21 use products only.
6	hadamard_bool in the chain	resolved	Algorithm 4.132 passes hadamard = ($m \neq 1$) explicitly.
7	Torsion basis slot convention	resolved	§4.4.2 states the (x_R, x_{RS}, x_S) permutation; Algorithm 4.94 line 31, Algorithm 4.95 line 14.
8	Zero-hint fallback work bound	clarify	Bound and failure policy requested in §26; no nonterminating v3 input established.
9	Matrix storage convention	resolved	Eq. (1), Algorithm 3.7, Chapter 6 encoding order.
10	2^{f-e} scaling in CHANGEOfBASIS	resolved	Algorithm 3.8 defines $\mathcal{B}_2 = \mathcal{B}_1 \cdot 2^{f-e} M$ with entries in $[0, 2^e)$; Algorithm 3.9 reduces first.
11	Cubical Tate third-argument sign	resolved	Note after Algorithm 4.103; Algorithm 4.106 inverts ζ_3, ζ_4 explicitly.
12	Matrix exponent omits HD_extra_torsion	resolved	Algorithm 3.23 line 18 uses $e_{\text{rsp}} + 2$.
13	Normalization of $(A_{24} : C_{24})$	resolved	Algorithm 4.94, Algorithm 4.95 require $(A : 1)$; Algorithm 4.99 ends in NORMALIZEMONTGOMERY.
14	Gluing requires Jacobian y	moot	Superglue gluing is x -only (Algorithm 4.122 to Algorithm 4.124).
15	Jacobian doubling for the strategy	moot	Algorithm 4.114 doubles theta points.
16	DIFFERENCEPOINT normalization factor	resolved	Algorithm 4.92 line 4 states $\gamma = C(\overline{C} \overline{X_P} Z_Q - Z_P \overline{X_Q})^2$.
17	Never recompute $P - Q$	partial	Algorithm 4.150 returns a triple; Algorithm 3.21 pushes only two points (§8).
18	Change-of-basis application convention	resolved	Eq. (1), Algorithm 3.7.
19	RANDOMIDEALGIVENNORM β range inclusive	resolved	Algorithm 4.82 with Algorithm 4.24 closed intervals; byte cost now differs (§6).
20	GENERALIZEDREPRESENT INTEGER ranges	moot	Algorithm 4.46 samples $z, t \geq 1$ in $\mathcal{O}_0$ with no isogeny condition; new divergences in §6.

#	v2.0.1 finding	Status	v3.0 locus
21	Response lattice product vs. intersection	resolved	Algorithm 3.23 lines 14 and 15, Algorithm 3.18.
22	LATTICESAMPLING underspecified	partial	Dual box specified in Algorithm 4.83; distribution correction in §1 and draw/failure policy in §14.
23	L^2 deep-insertion order	moot	Floating-point LLL removed (§4.3.3).
24	FIXEDDEGREEISOGENY u^{-1}	moot	Removed; the scaling appears as v_1, c_2 in Algorithm 4.150.
25	Fixed-precision HNF	moot	Inert representation with bounds (Table 10).
26	Endomorphism image $P - Q$ and Okeya–Sakurai	moot	No Jacobian lift; Algorithm 4.150 line 9 pushes R_1 .
27	GETINDEXSPLITTING malformed input	resolved	Algorithm 4.128 raises; Algorithm 4.133 returns false.
28	Signing key caches the generator	moot	The key stores (N, x, y) .
29	IDEALTOISOGENY line 6 matrix formula	moot	Klopin removed; see §17.
30	β lives in the reduced ideal	resolved	Algorithm 4.149 line 20 maps β_1 back through γ .
31	SUITABLEIDEALS $v_2(\gcd)$	moot	Removed.
32	Algorithm 8.47 implementation variants	resolved	One ISOGENY22CHAIN; kernel order 2^{e+2} stated in §4.5.4.1 and Algorithm 3.20.
33	IDEALTOISOGENY output basis alignment	resolved	Algorithm 4.150 pushes $[v_1]\beta_1(\mathcal{B}_0)$ (§4.6.1).
34	IDEALTOISOGENY must propagate $P - Q$	resolved	Algorithm 4.150 output is a triple.
35	Splitting randomization	moot	Splitting is deterministic: Algorithm 4.126 keeps the largest of the six Montgomery coefficients. The v2 randomized representative-selection procedure is no longer used.
36	Multi-order β transport	moot	Removed.
37	Connecting-ideal norm encoding	moot	Removed.
38	Action-matrix order index	moot	Only $\mathcal{O}_0$ (Algorithm 4.134).
39	Sign error in Algorithm 4.9 line 11	moot	Algorithm 3.24 line 9 decodes the aux basis at $e_{\text{rsp}} + 2$ directly.
40	chl top-bit canonicity	resolved	$\lambda = 128$ fills 16 bytes; Algorithm 3.4.
41	Lower bound on e'_{rsp}	moot	Chain length is fixed at e_{rsp} .
42	Extra-torsion-free chain lower bound	moot	One chain variant, fixed lengths.

Totals: 22 resolved, 17 moot, 2 partial, and 1 clarification request.

Checks and source material

The exact arithmetic checks accompanying the security findings run with the Python standard library:

```
python3 scripts/audit_sqisign_v3_design.py
```

They cover interval widths 1–512 at three draw widths, the three EIBox calculations, the illustrative lattice rejection, degree-9 kernel enumeration, degree-weight comparisons, and the small-field matrix count. They establish the stated finite counts; they do not simulate H_{sim} or run an end-to-end signer or verifier.

The conformance sections retain the earlier implementation comparisons and their stated limits. This update corrects the affected sampler, hint, and verifier recommendations; it does not rerun all reference KATs or independently revalidate every inherited comparison.

The reviewed PDF is v3.0, dated 1 September 2026. Its SHA-256 digest is

843912e5eb3dee20288af9fc9f01c0e1f1055834d3ce87c4687795c13adfd73c.

Reference paths in the findings are relative to `src/` at commit [6d017708db40](#). Ideal and quaternion norms are reduced norms; basis matrices act column by column as in Eq. (1) of the specification.

For tracking alongside the working notes, *Biased interval sampling* has the link target [SQI3-SPEC-01](#), and *Hint degree weighting* has [SQI3-PROOF-02](#).

References

- [1] SQIsign team. *SQIsign: Algorithm specifications and supporting documentation, version 3.0*, 1 September 2026. Sampler p. 28; bounded response sampling p. 51; basis hints p. 58; Figure 4 p. 104; stronger security and timing pp. 110–111.
- [2] SQIsign team. *the-sqisign, round-3 reference implementation*, tag `nist-v3`, commit [6d017708db40](#).
- [3] SQIsign team. *Reference integer sampler*, `src/mp/ref/lvlx/mp.c`, `ibz_rand_interval_with_domain`.
- [4] Aardal, Basso, De Feo, Patranabis, and Wesolowski. *A Complete Security Proof of SQIsign*, IACR ePrint 2025/379, consulted version dated 1 August 2025, Section 5.6.
- [5] SQIsign team. *Reference verifier*, `src/verification/ref/lvlx/verify.c`, `protocols_verify`.
- [6] SQIsign team. *Reference gluing validation*, `src/hd/ref/lvlx/gluing.c`, `verify_two_torsion` and `gluing_compute`.
- [7] D. Connolly. *SQIsign v2.0.1 Specification Review*, Selkie Cryptography, 2026; archived source.